

## On writing a design spec:

Hello friends. This is not a design specification, but rather a document on what you ought to include in a design specification, intended to help you make something nice. I've included a small checklist of what you may want to include to make it as useful as possible. So if you just want to check that your future design specs are up to spec (hehe), take a look in the bottom. Remember, this document can be practical, it's better you have a short informative document than a long slog. This is done most easily by following the **KISS** -acronym: **Keep It Simple, Stupid**

### What is a design spec?

A long time ago, hardware designers realized that it could be a good idea to be able to communicate what they had designed or what they were working on. This led to the incredibly sophisticated idea of documentation. One of these documents is the topic of today: namely the design specification. The design specification is a document that sets the requirements of a design. It describes what a **design is intended to be and do..**

This is actually quite useful. It makes the **RTL-writing** a lot more comfortable when you have a nice checklist of features to implement (recall how lovely the VLSI course was). Furthermore it gives you an expectation of what the device ought to do, which makes **verification much more straightforward.**

## What should it contain?

A hardware spec should answer the following questions::

- **Purpose** - What is the intention behind this design?
- **Functional Requirements** - What is this supposed to do?
- **Constraints and Limitations** - What are the constraints that apply to the design?
- **Interfacing** - How is the design accessed?
- **(Compliance)** - Should the design comply with any standards? University Students probably don't need to worry about this 😊

These points are really all we need to state and we're pretty much done. Let's take a bit more in depth into each category, with a small FIFO as an example design.

The **Purpose** is really easy, you state why you need the design in question. A simple example for a FIFO could be:

"The purpose of the FIFO is to act as a buffer for incoming data to the chip." Simply put, state why said design is needed in the first place and what objective it should achieve.

**Functional Requirements** comes next. This is best presented as a **list of things it should do**. It doesn't need to explain the exact details of how they are done, but it should describe the device's intended behavior. Remember, this document should be useful when you are constructing your design. Advanced details can be communicated in other documents. In the case of our FIFO:

"The FIFO is intended to implement a First In First Out buffer, or a simple queue. The following are a breakdown of its behaviors:"

- The FIFO **must** behave like a First In First Out Queue.
- The FIFO **must** always output the oldest entry.
- The FIFO **must** assert **full** when it is full.
- The FIFO **must** assert **empty** when it is empty.
- The FIFO **should** notify when it is almost empty. (1 entry)
- The FIFO **should** notify when it is almost full. (Size-1 entries)
- The FIFO **must** only store data when **write\_en** is asserted high.
- The FIFO **must** only "pop" an element when **read\_en** is asserted high.
- The FIFO **must** support simultaneous read/writes, given that the fifo is not empty.

As you can see, the design has been broken down into a set of **musts** and **shoulds**. Let's go over that distinction. A **must** is something that is integral to the design and must be true. A **should** is something that the design could survive without, but could be nice/useful. This distinction is nice to have as it allows you to plan more realistically. However, these bullet points look ugly. Fortunately we can take the same points and put them into a more digestible table:

|        |                                                                     |
|--------|---------------------------------------------------------------------|
| Must   | behave like a First In First Out Queue.                             |
| Must   | always output the oldest entry.                                     |
| Must   | assert <b>full</b> when it is full.                                 |
| Must   | assert <b>empty</b> when it is empty.                               |
| Must   | only store data when <b>write_en</b> is asserted high               |
| Must   | only “pop” an element when <b>read_en</b> is asserted high.         |
| Must   | support simultaneous read/writes, given that the fifo is not empty. |
| Should | notify when it is almost empty. (1 entry)                           |
| Should | notify when it is almost full. (Size-1 entries)                     |

In this case, a helpful graph could also be useful for describing FIFO behaviour in general. Note: there are a few signals described, although you don't have to name them, it could be prudent. In that case consider introducing the interfacing before the functional requirements.

**Constraints and Limitations** are typically used to clarify things such as dimensions and set limits on what can be expected. This does not need to be exhaustive in the same sense, but should be sufficient so that there are no points of contention. In other words, you do not need to let the reader know that the FIFO isn't capable of space travel, but perhaps you should let them know it should be larger than 1 element and smaller than 128. Let common sense prevail here. Think of these as addendums and corners.

In the case of our FIFO:

- The FIFO must store at least 8 elements
- The FIFO must have a bitwidth of 32.
- The FIFO should not have to block writes when full or reads when empty, surrounding control logic is responsible for this.

**Interfacing** is surprisingly easy for us hardware people, we just describe which signals are expected to exist and how they are expected to be used together. If the interfacing sequencing is complex, don't be afraid to describe them via waveforms.

In the case of our FIFO it could look something like this:

"The FIFO consists of 2 main data signals along with control signals. These are as follows:

- **queue**[31:0]
- **dequeue**[31:0]
- **full**
- **empty**
- **write\_en**
- **read\_en**
- **almost\_full** (tentative)
- **almost\_empty** (tentative)
- **clock**
- **reset**

Asserting **write\_en** high will insert the value of **queue** into the fifo and increment the write pointer in the next clock cycle. Asserting **read\_en** high will modify value on **deque** after the next positive clock edge."

With this we have something that resembles a very simple design spec. It covers why the design exists, what it should be doing, how it's connected to the outside world, and some of its limitations. Naturally, this could be expanded upon quite a lot, but I wanted to keep this document short so that you will read the whole thing.

You can now make simple design specs, and as an added bonus you can look at an ASCII cat:

```
| \ - - - / |  
|  o _o  |  
| \ ^ _ / |
```